

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 数据结构

项目名称： 二叉树的遍历与图的遍历

2023年春季学期

上机时间：第 15 周 星期 四 第 5 ~ 8 节

学生姓名： 张清晨 专业班级： 计算机 2101 班

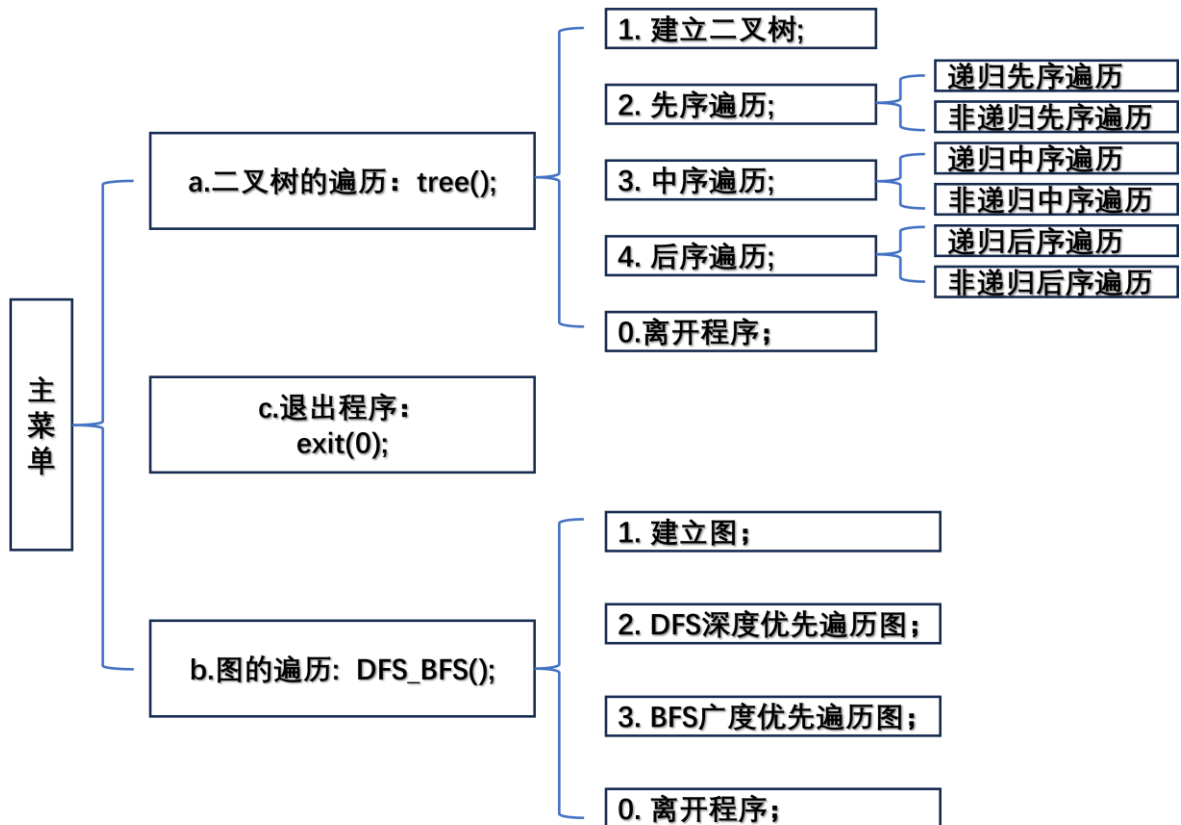
指导教师： 孙焕良 成 绩：

项目名称	二叉树的遍历与图的遍历	学时	4
日期：2023 年 6 月 15 日		第 15 教学周	
硬件及软件			
名 称		版本、型号	
Visual C++		6.0 及以上	
上机目的与要求： 1、设计并实现二叉树的先序、中序和后序遍历操作； 2、设计一个结点数不少于 50 的连通图，实现图的遍历算法； 3、要求功能齐全； 4、有菜单，显示友好； 5、报告第 1-2 页、最后一页及部分截图。 上机任务： 1、题目； 2、画出基本结构及操作原理示意图； 3、程序清单：要示对类型定义、主菜单、函数定义等内容给出注释。 4、部分结果截图：可打印。 5、结果分析和总结：最后一页			

一、 题目简介

设计并实现二叉树的先序、中序和后序遍历操作（递归和非递归）；
实现图的遍历算法：深度优先遍历和广度优先遍历。

二、 程序实现流程图和基本结构及操作原理示意图



程序实现流程图

三、 程序清单

```
/*
数据结构上机 2   2023-6-15   计算机 2101 张清晨
*/

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <malloc.h>
#include <stdlib.h>

#define OK          1
#define ERROR       0
#define TRUE        1
#define FALSE       0
#define OVERFLOW    -1

#define STACK_INIT_SIZE    100 //栈初始开辟长度
#define STACKINCREMENT     10  //栈空间满时的增量

#define INFINITY 32767
#define MAXVEX 60
int visited[MAXVEX]; /* Node: visited flag with dynamic array, good idea ! */

typedef int Status;
typedef char ElemType; // 二叉树结点元素类型
typedef int QElemType;
typedef int VertexType; //顶点数据类型
typedef int EdgeType; //边表的权值类型

typedef struct BiTNode{ // 二叉树结点结构
    char data;           // 结点数据
    struct BiTNode *lchild; // 左孩子
    struct BiTNode *rchild; // 右孩子
}BiTNode,*BiTree;
typedef BiTree SElemType;

typedef struct{//栈结构定义
    SElemType *base; //栈底指针
    SElemType *top;  //栈顶指针
    int stacksize;
}SqStack;
```

```

typedef struct QNode{
    QElemType data;
    struct QNode *next;
}QNode, *QueuePtr;

typedef struct{
    QueuePtr front;
    QueuePtr rear;
} LinkQueue;

typedef struct graph{
    VertexType data[MAXVEX]; //图的顶点
    EdgeType Edge[MAXVEX][MAXVEX]; //图的边表
    int NumVertex,NumEdge; //图的顶点数与边数
}Graph;

Status InitStack(SqStack *S);
//构造一个空栈 S
Status DestroyStack(SqStack *S);
//销毁栈 S, S 不再存在
Status ClearStack(SqStack *S);
//把栈 S 置为空栈
Status StackEmpty(SqStack S);
//若栈 S 为空栈, 则返回 TRUE, 否则返回 FALSE
int StackLength(SqStack S);
//返回 S 元素的个数, 即栈的长度
Status GetTop(SqStack S,SElemType *e);
//若栈不为空, 则用 e 返回 S 的栈顶元素, 并返回 OK; 否则返回 FALSE
Status Push(SqStack *S,SElemType e);
//插入元素 e 为新的栈顶元素
Status Pop(SqStack *S,SElemType *e);
//若栈 S 不为空, 则删除 S 的栈顶元素, 用 e 返回其值, 并返回 OK,否则返回 ERROR
Status StackTraverse(const SqStack *S);
//从栈底到栈顶依次对每个元素进行访问
void tree();
BiTree CreateBiTree(BiTree T);
// 按先后次序输入二叉树中结点的值 (一个字符), 空格表示空树
// 构造二叉链表表示的二叉树 T
Status PreOrderRecursionTraverse(BiTree T);
// 采用二叉链表存储结构, Visit 是对数据元素操作的应用函数
// 先序遍历二叉树 T 的递归算法, 对每个数据元素调用函数 Visit
Status InOrderRecursionTraverse(BiTree T);
// 采用二叉链表存储结构, Visit 是对数据元素操作的应用函数
// 中序遍历二叉树 T 的递归算法, 对每个数据元素调用函数 Visit
Status PostOrderRecursionTraverse(BiTree T);
// 采用二叉链表存储结构, Visit 是对数据元素操作的应用函数
// 后序遍历二叉树 T 的递归算法, 对每个数据元素调用函数 Visit

```

```

Status PreOrderNonRecursionTraverse(BiTree T);
// 采用二叉链表存储结构，Visit 是对数据元素操作的应用函数
// 先序遍历二叉树 T 的非递归算法，对每个数据元素调用函数 Visit
Status InOrderNonRecursionTraverse(BiTree T);
// 采用二叉链表存储结构，Visit 是对数据元素操作的应用函数
// 中序遍历二叉树 T 的非递归算法，对每个数据元素调用函数 Visit
Status PostOrderNonRecursionTraverse(BiTree T);
// 采用二叉链表存储结构，Visit 是对数据元素操作的应用函数
// 后序遍历二叉树 T 的非递归算法，对每个数据元素调用函数 Visit
Status Visit(ElemType e);
// 对二叉树中的数据元素访问


void DFS_BFS(); //图的两种遍历
void CreateGraph(Graph *G); //创建图
int Firstvex_Graph(Graph G, int k); //返回 k 的第 1 个邻接点
int Nextvex_Graph(Graph G, int i, int j); //返回 i 的第 j 个邻接点的下一个邻接点
void DFS(Graph G, int k); //DFS 遍历
void BFS(Graph G); //BFS 遍历
Status InitQueue(LinkQueue *Q); //Initialize an empty list
Status QueueEmpty(LinkQueue *Q);
Status EnQueue(LinkQueue *Q, QElemType e);
QElemType DeQueue(LinkQueue *Q);


int main()
{
    char c;

    while(1){ //主菜单
        printf("请选择以下三种操作之一： \n");
        printf("a.二叉树的遍历\n");
        printf("b.图的遍历\n");
        printf("c.离开\n");

        scanf(" %c",&c);

        switch(c)
        {
            case 'a':
                tree(); //二叉树的遍历
                break;
            case 'b':
                DFS_BFS(); //图的遍历
                break;
            case 'c':
                exit(0);
            default:
                printf("Print an incorrect letter;\n");
                break;
        }
    }
}

```

```

    }
    system("PAUSE");
    return 0;

}
}

//二叉树的遍历
void tree()
{
    BiTree T=NULL;
    char c;

    while(1){
        printf("二叉树的遍历\n");
        printf("=====\n");
        printf("1. 建立二叉树;    2. 先序遍历;\n");
        printf("3. 中序遍历;        4. 后序遍历;    0.离开程序; \n");
        printf("=====\n");

        scanf(" %c",&c);
        //  getchar();

        switch(c){
            case '1':
                printf("请按先序遍历输入二叉树元素（每个结点一个字符，空结点为'#'，如 ABC##DE#G##F###）:\n");//如 ABC##DE#G##F###
                T=CreateBiTree(T);
                if(T) printf("创建成功! \n\n");
                else printf("创建失败! \n\n");
                break;
            case '2':
                printf("递归先序遍历: ");
                PreOrderRecursionTraverse(T);
                printf("\n 非递归先序遍历: ");
                PreOrderNonRecursionTraverse(T);
                printf("\n\n");
                break;
            case '3':
                printf("递归中序遍历: ");
                InOrderRecursionTraverse(T);
                printf("\n 非递归中序遍历: ");
                InOrderNonRecursionTraverse(T);
                printf("\n\n");
                break;

```

```

        case '4':
            printf("递归后序遍历: ");
            PostOrderRecursionTraverse(T);
            printf("\n 非递归后序遍历: ");
            PostOrderNonRecursionTraverse(T);
            printf("\n\n");
            break;
        case '0':
            exit(0);
        default:
            printf("Print an incorrect letter;\n");
            break;
    }

}

system("PAUSE");
}

// 按先后次序输入二叉树中结点的值（一个字符），空格表示空树
// 构造二叉链表表示的二叉树 T
BiTree CreateBiTree(BiTree T)
{
    char ch;
    scanf(" %c",&ch);
    if(ch=='#') T=NULL;
    else{
        if(!(T=(BiTNode *)malloc(sizeof(BiTNode))))
            exit(OVERFLOW);
        T->data=ch;           // 生成根节点
        T->lchild=CreateBiTree(T->lchild);    // 构造左子树
        T->rchild=CreateBiTree(T->rchild);    // 构造右子树
    }
    return T;
}

// 先序遍历递归算法
Status PreOrderRecursionTraverse(BiTree T)
{
    if(T){
        if(!Visit(T->data)) return ERROR;
        PreOrderRecursionTraverse(T->lchild);
        PreOrderRecursionTraverse(T->rchild);
    }
    return OK;
}

```


// 中序遍历递归算法

```
Status InOrderRecursionTraverse(BiTree T)
{
    if(T){
        InOrderRecursionTraverse(T->lchild);
        if(!Visit(T->data)) return ERROR;;
        InOrderRecursionTraverse(T->rchild);
    }
    return OK;
}
```

//后序遍历递归算法

```
Status PostOrderRecursionTraverse(BiTree T)
{
    if(T){
        PostOrderRecursionTraverse(T->lchild);
        PostOrderRecursionTraverse(T->rchild);
        if(!Visit(T->data)) return ERROR;;
    }
    return OK;
}
```

// 先序遍历二叉树 T 的非递归算法

```
Status PreOrderNonRecursionTraverse(BiTree T)
{
    SqStack S;
    SElemType p;
    InitStack(&S);
    Push(&S, T);
    while (!StackEmpty(S))
    {
        Pop(&S, &p);
        if (!Visit(p->data)) return ERROR;
        if (p->rchild) Push(&S, p->rchild);
        if (p->lchild) Push(&S, p->lchild);
    }
    return OK;
}
```

// 中序遍历二叉树 T 的非递归算法

```
Status InOrderNonRecursionTraverse(BiTree T)
{
    SqStack S;
    SElemType p;
    InitStack(&S);
    p = T;
    while (p || !StackEmpty(S))
    {
```

```

        if (p)
        {
            Push(&S, p);
            p = p->lchild;
        }
        else
        {
            Pop(&S, &p);
            Visit(p->data);
            p = p->rchild;
        }
    }
    return OK;
}

// 后序遍历二叉树 T 的非递归算法
Status PostOrderNonRecursionTraverse(BiTree T)
{
    SqStack S;
    SElemType p,q;
    InitStack(&S);      Push(&S,T); // 根指针入栈
    while(!StackEmpty(S)){
        while(GetTop(S, &p) && p && (p -> lchild || p -> rchild)){
            Push(&S, p -> rchild);
            Push(&S, p -> lchild);
        }
        if (!StackEmpty(S)){
            Pop(&S, &p);
            if (p){
                if (!Visit(p -> data)) return ERROR;
            }
            else{
                Pop(&S, &p);
                if (!Visit(p -> data)) return ERROR;
            }
            while (GetTop(S, &q) && q && p == q -> rchild){
                Pop(&S, &p);
                if (!Visit(p -> data)) return ERROR;
                GetTop(S, &q);
            }
        }
    }
    DestroyStack(&S);
    return OK;
}

```

```

// 对二叉树中的数据元素访问
Status Visit(ElemType e)
{
    if(e=='\0'){
        return ERROR;
    }else{
        printf("%c",e);
    }
    return OK;
}

//-----顺序栈操作-----

Status InitStack(SqStack *S){
    //构造一个空栈 S
    S->base=(SElemType *)malloc(STACK_INIT_SIZE*sizeof(SElemType));
    if(!S->base)//分配失败
    {
        printf("分配内存失败.\n");
        exit(0);
    }
    S->top=S->base;
    S->stacksize=STACK_INIT_SIZE;
    return OK;
}

Status DestroyStack(SqStack *S){
    //销毁栈 S, S 不再存在
    if(!S)//S 为空
    {
        printf("指针为空, 释放失败.\n");
        exit(0);
    }
    free(S->base);
    return OK;
}

Status ClearStack(SqStack *S){
    //把栈 S 置为空栈
    if(!S)//S 不存在
        return FALSE;
    S->top=S->base;//直接将栈顶指针指向栈底
    return OK;
}

```

```

Status StackEmpty(SqStack S){
    //若栈 S 为空栈，则返回 TRUE，否则返回 FALSE
    if(S.top==S.base)
        return TRUE;
    else
        return FALSE;
}

int StackLength(SqStack S){
    //返回 S 元素的个数，即栈的长度
    return S.stacksize;
}

Status GetTop(SqStack S,SElemType *e){
    //若栈不为空，则用 e 返回 S 的栈顶元素，并返回 OK；否则返回 FALSE
    if(S.top==S.base){
        return FALSE;
    }else{
        *e=*(S.top-1);
        return OK;
    }
}

Status Push(SqStack *S,SElemType e){
    //插入元素 e 为新的栈顶元素
    if(S->top-S->base>=S->stacksize){//栈已满，追加存储空间
        S->base=(SElemType
*)realloc(S->base,(S->stacksize+STACKINCREMENT)*sizeof(SElemType));
        if(!S->base)
        {
            printf("重新申请空间失败.\n");
            exit(0);
        }
        S->top=S->base+S->stacksize;//更改栈顶指针
        S->stacksize+=STACKINCREMENT;
    }
    *S->top++=e;
    return OK;
}

Status Pop(SqStack *S,SElemType *e){
    //若栈 S 不为空，则删除 S 的栈顶元素，用 e 返回其值，并返回 OK,否则返回 ERROR
    if(S->top==S->base){//栈为空
        return ERROR;
    }
    *e=*(--S->top);
    return OK;
}

```

```

//-----遍历图操作-----
void DFS_BFS(){
    Graph G;    //visited = (int *)malloc(G.NumVertex * sizeof(int));
    char c;

    while(1){
        printf("图的遍历\n");
        printf("=====\\n");
        printf("1. 建立图;                2. DFS 深度优先遍历图; \\n");
        printf("3. BFS 广度优先遍历图;    0. 离开程序; \\n");
        printf("=====\\n");

        scanf(" %c",&c);//flush(stdin);

        if (c == '1'){
            CreateGraph(&G);
            printf("建图完成! \\n");
        }
        else if (c == '2'){
            for(int i = 0; i < G.NumVertex; i++) visited[i] = 0;
            printf("Dfs:");
            DFS(G, -1);
        }
        else if (c == '3'){
            for(int i = 0; i < G.NumVertex; i++) visited[i] = 0;
            printf("Bfs:");
            BFS(G);
        }
        else if (c == '0')
            return;
        else
            printf("Print an incorrect letter;\\n");
        printf("\\n\\n");
    }
}

//创建图
void CreateGraph(Graph *G)
{
    int i,j,k;

    printf("请输入图的顶点数和边数:\\n");
    scanf("%d %d", &G->NumVertex, &G->NumEdge);
    printf("请输入图的顶点信息:\\n");
    for(i = 0;i < G->NumVertex; ++i)
    {
        //flush(stdin); //清空输入缓冲区，为了确保不影响后面的数据读取
        scanf("%d", &G->data[i]); //输入顶点的信息
    }
}

```

```

for(i = 0; i < G->NumVertex; ++i) //初始化图的权值为无限大
    for(j = 0; j < G->NumVertex; ++j)
        G->Edge[i][j] = INFINITY;
for(k = 0; k < G->NumEdge; ++k)
{
    int i, j, w;
    printf("请输入边的连接信息(vi,vj)和边的权值:\n");
    //fflush(stdin);
    scanf("%d%d%d", &i, &j, &w);
    G->Edge[i-1][j-1] = w; //边的权值
    G->Edge[j-1][i-1] = G->Edge[i-1][j-1]; //无向图存在反向链接,边的权值相同
}
}

//返回 k 的第 1 个邻接点
int Firstvex_Graph(Graph G, int k)
{
    int i;
    if(k >= 0 && k < G.NumVertex)
        for(i = 0; i < G.NumVertex; i++)
            if(G.Edge[k][i] != INFINITY)
                return i;
    return -1;
}

//返回 i 的第 j 个邻接点的下一个邻接点
int Nextvex_Graph(Graph G, int i, int j)
{
    int k;
    if(i >= 0 && i < G.NumVertex && j >= 0 && j < G.NumVertex)
        for(k = j + 1; k < G.NumVertex; k++){
            if(G.Edge[i][k] != INFINITY)
                return k;
        }
    return -1;
}

//DFS 遍历
void DFS(Graph G, int k)
{
    //printf("%d\n", k);
    int i;
    if(k == -1)
    {
        for(i = 0; i < G.NumVertex; i++)
            if(!visited[i])
                DFS(G, i);
    }
}

```

```

else
{
    visited[k] = 1;
    printf("%d ", G.data[k]);
    for(i = Firstvex_Graph(G, k); i >= 0; i = Nextvex_Graph(G, k, i))
        if(!visited[i])
            DFS(G, i);
}
}

//BFS 遍历
void BFS(Graph G)
{
    int i, j, k;
    LinkQueue Q;
    InitQueue (&Q);
    //printf("QWQWQWQ");
    for(i = 0; i < G.NumVertex; i++)
        if(!visited[i])
        {
            visited[i] = 1;
            printf("%d ", G.data[i]);
            EnQueue(&Q, i);
            while(!QueueEmpty(&Q))//qu 非空
            {
                k = DeQueue(&Q);
                for(j = Firstvex_Graph(G, k); j >= 0; j = Nextvex_Graph(G, k, j))
                    if(!visited[j])
                    {
                        visited[j] = 1;
                        printf("%d ", G.data[j]);
                        EnQueue(&Q, j);
                    }
            }
        }
}

```

//-----队列操作-----

Status InitQueue(LinkQueue *Q)

```
{
    Q -> front = Q -> rear = (QueuePtr)malloc(sizeof(QNode));
    if (Q -> front == NULL) exit(OVERFLOW);
    Q -> front -> next = NULL;

    return OK;
}
```

Status EnQueue(LinkQueue *Q, QElemType e)

```
{
    QueuePtr p;
    p = (QueuePtr)malloc(sizeof(QNode));
    if (!p) exit(OVERFLOW);
    p -> data = e;
    p -> next = NULL;
    Q -> rear -> next = p;
    Q -> rear = p;
    return OK;
}
```

Status QueueEmpty(LinkQueue *Q)

```
{
    if (Q -> rear == Q -> front) return ERROR;
    return OK;
}
```

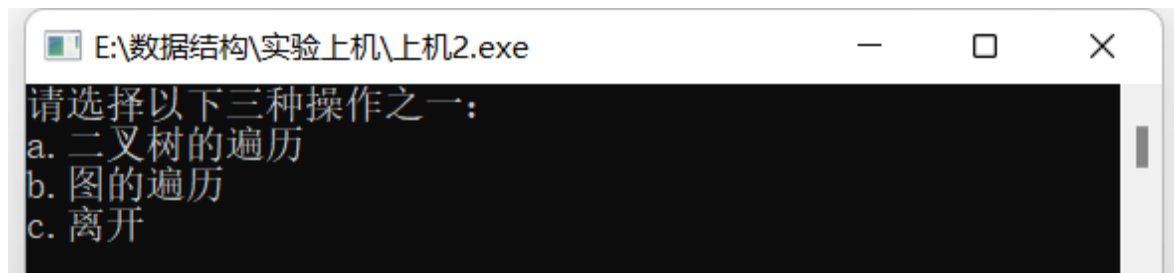
QElemType DeQueue(LinkQueue *Q)

```
{
    if (Q -> front == Q -> rear) return ERROR;

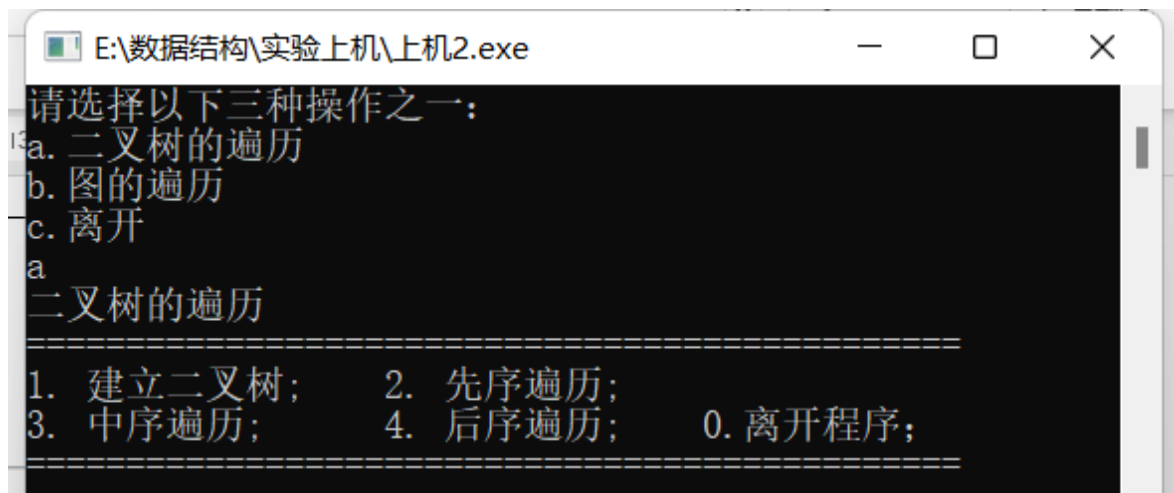
    QueuePtr p;
    p = Q -> front -> next;
    QElemType e = p -> data;
    Q -> front -> next = p -> next;
    if (Q -> rear == p)
        Q -> rear = Q -> front;
    free(p);
    return e;
}
```


四、 运行截图

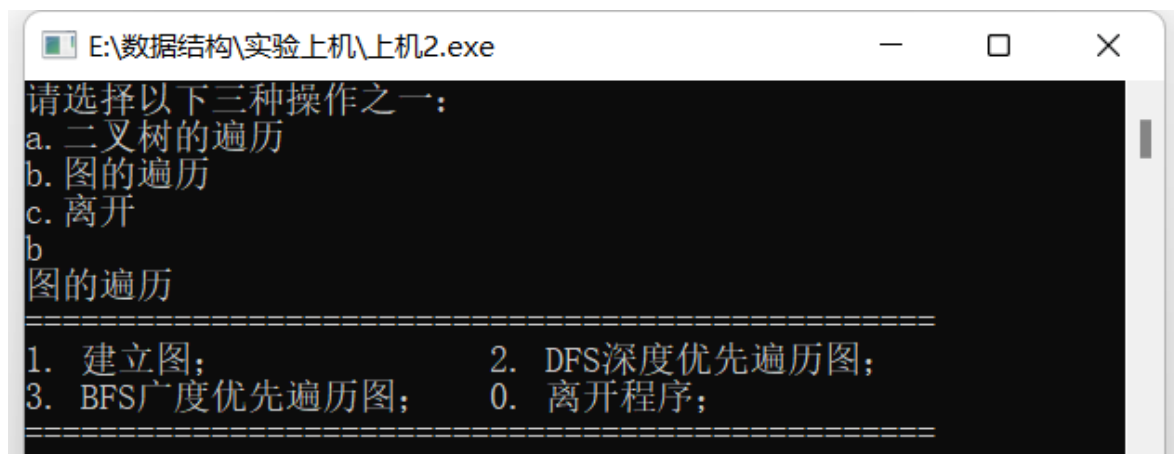
1. 主菜单



2. 二叉树遍历菜单



3. 图遍历菜单



4. 建立二叉树

```
E:\数据结构\实验上机\上机2.exe
请选择以下三种操作之一：
a. 二叉树的遍历
b. 图的遍历
c. 离开
a
二叉树的遍历
=====
1. 建立二叉树；    2. 先序遍历；
3. 中序遍历；      4. 后序遍历；    0. 离开程序；
=====
1
请按先序遍历输入二叉树元素（每个结点一个字符，空结点为'#'，如ABC##DE#G##F###）：
ABC##DE#G##F###
创建成功!
```

5. 先序遍历

```
E:\数据结构\实验上机\上机2.exe
2
递归先序遍历: ABCDEGF
非递归先序遍历: ABCDEGF
```

6. 中序遍历

```
E:\数据结构\实验上机\上机2.exe
3
递归中序遍历: CBEGDFA
非递归中序遍历: CBEGDFA
```

7. 后序遍历

```
E:\数据结构\实验上机\上机2.exe
4
递归后序遍历: CGEFDDBA
非递归后序遍历: CGEFDDBA
```

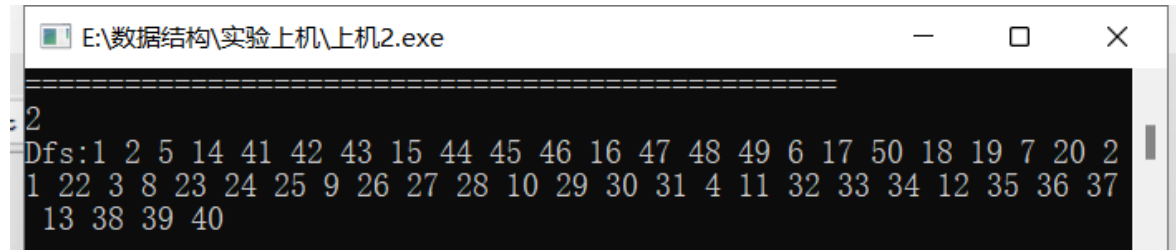
8. 图的建立

```
E:\数据结构\实验上机\上机2.exe
=====
:1
请输入图的顶点数和边数:
50 51
请输入图的顶点信息:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
48 49 50
请输入边的连接信息(vi, vj)和边的权值:
1 2 1
请输入边的连接信息(vi, vj)和边的权值:
1 3 1
请输入边的连接信息(vi, vj)和边的权值:
1 4 1
请输入边的连接信息(vi, vj)和边的权值:
2 5 1
请输入边的连接信息(vi, vj)和边的权值:
2 6 1
请输入边的连接信息(vi, vj)和边的权值:
2 7 1
请输入边的连接信息(vi, vj)和边的权值:
3 8 1
请输入边的连接信息(vi, vj)和边的权值:
3 9 1
请输入边的连接信息(vi, vj)和边的权值:
3 10 1
请输入边的连接信息(vi, vj)和边的权值:
4 11 1
请输入边的连接信息(vi, vj)和边的权值:
4 12 1
请输入边的连接信息(vi, vj)和边的权值:
4 13 1
请输入边的连接信息(vi, vj)和边的权值:
5 14 1
请输入边的连接信息(vi, vj)和边的权值:
5 15 1
请输入边的连接信息(vi, vj)和边的权值:
5 16 1
请输入边的连接信息(vi, vj)和边的权值:
6 17 1
请输入边的连接信息(vi, vj)和边的权值:
6 18 1
请输入边的连接信息(vi, vj)和边的权值:
6 19 1
请输入边的连接信息(vi, vj)和边的权值:
7 20 1
请输入边的连接信息(vi, vj)和边的权值:
7 21 1
请输入边的连接信息(vi, vj)和边的权值:
```

E:\数据结构\实验上机\上机2.exe

```
10 29 1
请输入边的连接信息(vi, vj)和边的权值:
10 30 1
请输入边的连接信息(vi, vj)和边的权值:
10 31 1
请输入边的连接信息(vi, vj)和边的权值:
11 32 1
请输入边的连接信息(vi, vj)和边的权值:
11 33 1
请输入边的连接信息(vi, vj)和边的权值:
11 34 1
请输入边的连接信息(vi, vj)和边的权值:
12 35 1
请输入边的连接信息(vi, vj)和边的权值:
12 36 1
请输入边的连接信息(vi, vj)和边的权值:
12 37 1
请输入边的连接信息(vi, vj)和边的权值:
13 38 1
请输入边的连接信息(vi, vj)和边的权值:
13 39 1
请输入边的连接信息(vi, vj)和边的权值:
13 40 1
请输入边的连接信息(vi, vj)和边的权值:
14 41 1
请输入边的连接信息(vi, vj)和边的权值:
14 42 1
请输入边的连接信息(vi, vj)和边的权值:
14 43 1
请输入边的连接信息(vi, vj)和边的权值:
15 44 1
请输入边的连接信息(vi, vj)和边的权值:
15 45 1
请输入边的连接信息(vi, vj)和边的权值:
15 46 1
请输入边的连接信息(vi, vj)和边的权值:
16 47 1
请输入边的连接信息(vi, vj)和边的权值:
16 48 1
请输入边的连接信息(vi, vj)和边的权值:
16 49 1
请输入边的连接信息(vi, vj)和边的权值:
17 50 1
请输入边的连接信息(vi, vj)和边的权值:
17 51 1
请输入边的连接信息(vi, vj)和边的权值:
17 52 1
建图完成!
```

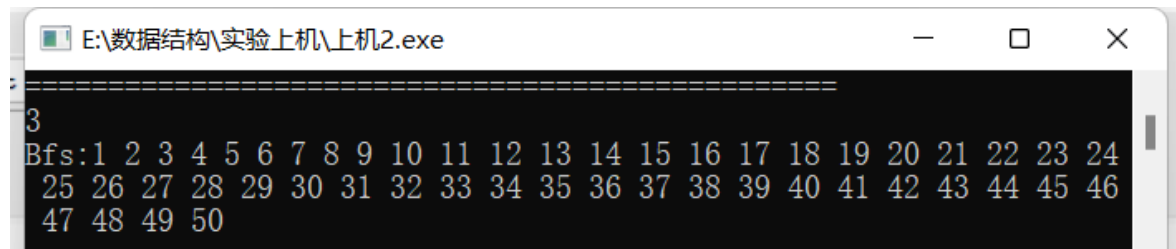
9. DFS 深度优先遍历图



The screenshot shows a command prompt window titled "E:\数据结构\实验上机\上机2.exe". The output displays a DFS traversal sequence starting from node 2. The sequence is: 2, 1, 22, 3, 8, 23, 24, 25, 9, 26, 27, 28, 10, 29, 30, 31, 4, 11, 32, 33, 34, 12, 35, 36, 37, 13, 38, 39, 40. The output is formatted with a separator line of equals signs above the sequence.

```
E:\数据结构\实验上机\上机2.exe
=====
2
Dfs:1 2 5 14 41 42 43 15 44 45 46 16 47 48 49 6 17 50 18 19 7 20 2
1 22 3 8 23 24 25 9 26 27 28 10 29 30 31 4 11 32 33 34 12 35 36 37
13 38 39 40
```

10.BFS 广度优先遍历图



The screenshot shows a command prompt window titled "E:\数据结构\实验上机\上机2.exe". The output displays a BFS traversal sequence starting from node 3. The sequence is: 3, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50. The output is formatted with a separator line of equals signs above the sequence.

```
E:\数据结构\实验上机\上机2.exe
=====
3
Bfs:1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46
47 48 49 50
```

结果分析和总结：

通过第二次上机实验报告的书写，加深了我对二叉树遍历和图的遍历的理解与记忆，同时也掌握了：二叉树的遍历分为先序，中序和后序遍历，每种又可以分成递归和非递归遍历。图的遍历有两种方式，深度优化遍历和广度优化遍历。在代码的书写过程中我也遇到了很多问题、警告和报错，通过查阅相关资料及询问同学，最终都得以解决。同时我认为很重要的一点是要学会自己书写代码，而不能仅仅是能看懂然后对着书本敲！这一点我还需要加强。这部分的知识比较难懂，需要多加学习思考。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：